C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > core > autograd.py

```python
import pyopencl as cl

class AutogradContext:
    """
    Единый менеджер графа вычислений, привязанный к конкретной очереди OpenCL.
    Устраняет разрозненность стеков у разных тензоров.
    """
    def __init__(self, queue: cl.CommandQueue):
        self.queue = queue
        self.tape = []          # Единая лента (стек) для backward-колбэков
        self.is_training = True # Флаг режима обучения (запись в tape)

    def register_backward(self, backward_fn):
        """Добавляет операцию в общую ленту, если включен режим обучения"""
        if self.is_training:
            self.tape.append(backward_fn)

    def run_backward(self):
        """Последовательно разворачивает всю ленту от конца к началу"""
        while self.tape:
            backward_fn = self.tape.pop()
            backward_fn()
        # Гарантируем, что все backward-ядра на GPU завершили работу
        self.queue.finish()

    def zero_grad(self, params):
        """Безопасно зануляет буферы градиентов для списка тензоров"""
        for param in params:
            if param.grad_buffer is not None:
                cl.enqueue_fill_buffer(
                    self.queue,
                    param.grad_buffer,
                    param.dtype.type(0.0),
                    0,
                    param.nbytes
                )
        self.queue.finish()
```

Ты допустил небольшую ошибку:

```
                    self._captured_inputs.add(t)

        # --- СБОРКА ФИНАЛЬНОГО ТЕКСТА КЕРНЕЛА FORWARD ---
        kernel_hash = abs(hash(program_text))
        kernel_name = f"text_forward_{kernel_hash}"

        kernel_args = []
        cl_buffers_call_fwd = []
        max_elements = 1

        for tensor_name in sorted(list(detected_tensors_set)):
            tensor_obj = caller_locals[tensor_name]
            if tensor_obj.size > max_elements:
                max_elements = tensor_obj.size

            kernel_args.append(f"__global float* {tensor_name}")
            kernel_args.append(f"const int {tensor_name}Size")
            cl_buffers_call_fwd.append(tensor_obj.data_buffer)
            cl_buffers_call_fwd.append(np.int32(tensor_obj.size))

            # Если обучаемся, сразу добавляем указатели на буферы градиентов в ар
            if self.is_training:
                kernel_args.append(f"__global float* grad_{tensor_name}")
                cl_buffers_call_fwd.append(tensor_obj.grad_buffer)

        args_str = ", ".join(kernel_args)
        forward_statements_str = "\n    ".join(compiled_statements)

        full_forward_code = f"""
{BASE_CL_FUNCTIONS}

__kernel void {kernel_name}({args_str}) {{
    int tid = {self.warp_size} * get_global_id(0) + get_global_id(1);
    {forward_statements_str}
}}
"""
```

Ты предлогаешь собрать 1 большое ядро как для прямого так и для обратного! На ммой взгляд чтобы избежать ошибок и не передавать неиспользуемые аргументы, нам следует реализовать отдельное ядро для обучения kernel_args_backward.

Также актуальная версия кода для корректировки:

```python
class LocalRunner:
    def __init__(self, ctx, queue, is_training=False):
        self.ctx = ctx
        self.queue = queue
        self.is_training = is_training
        self.backward_stack = []
        # --- НОВЫЕ АТРИБУТЫ ДЛЯ СВЯЗИ СО СЛОЯМИ ВЫШЕ ---
        self._captured_inputs = set()   # Имена тензоров-аргументов (справа от =)
        self._captured_outputs = set()  # Имена изменившихся тензоров (слева от =)
        # Автодетекция размера варпа
        dummy_code = "__kernel void dummy() {}"
        dummy_prg = cl.Program(self.ctx, dummy_code).build()
        try:
            self.warp_size = dummy_prg.dummy.get_work_group_info(
                cl.kernel_work_group_info.PREFERRED_WORK_GROUP_SIZE_MULTIPLE,
                self.queue.device
            )
        except Exception:
            self.warp_size = 32

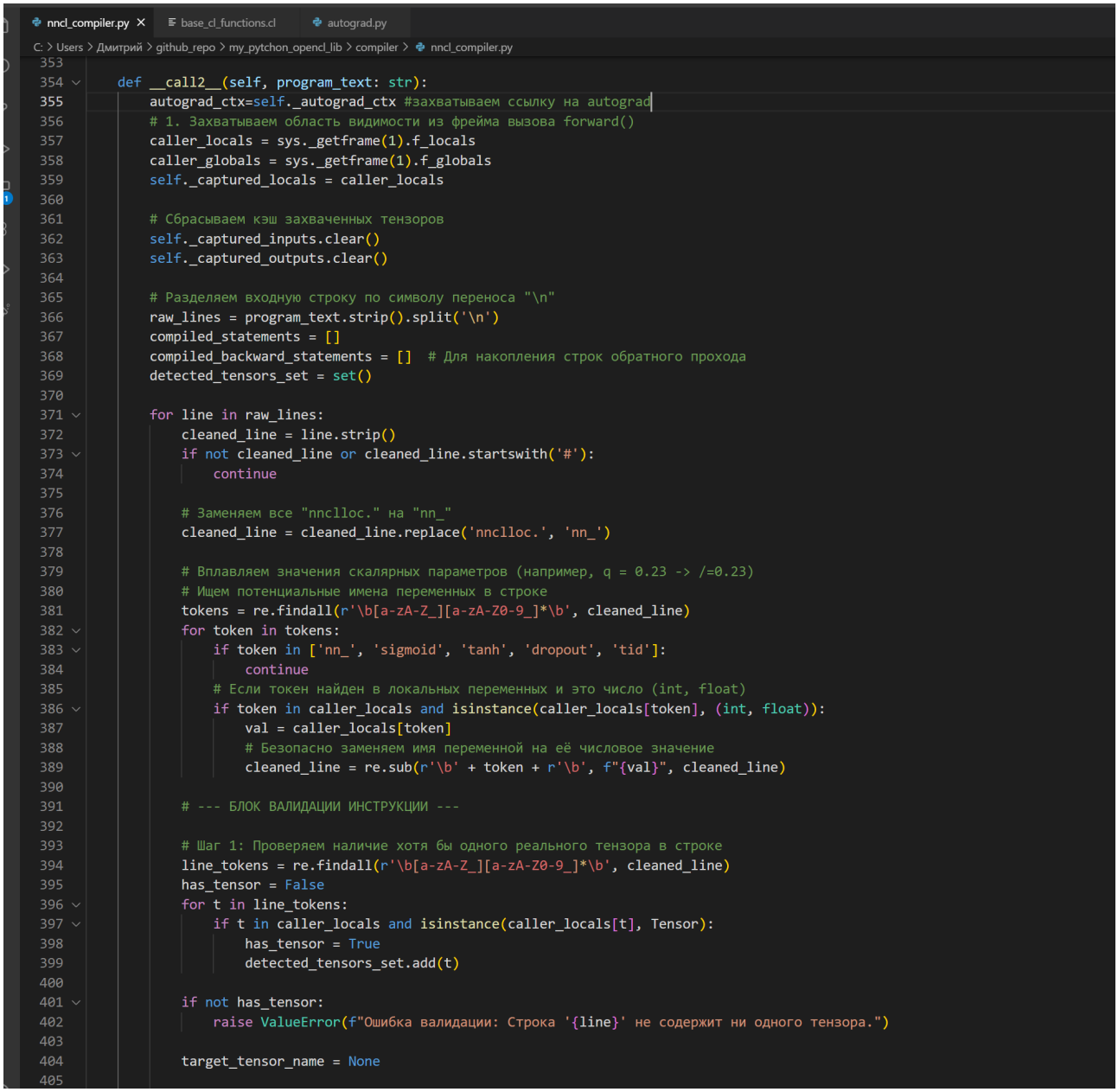
```

```
        return cls._cache[kernel_code]

class LocalRunner:
    def __init__(self, ctx, queue, is_training=False,autograd_ctx=None):
        self.ctx = ctx
        self.queue = queue
        self.is_training = is_training
        self.backward_stack = []
        # --- НОВЫЕ АТРИБУТЫ ДЛЯ СВЯЗИ СО СЛОЯМИ ВЫШЕ ---
        self._captured_inputs = set()   # Имена тензоров-аргументов (справа от =)
        self._captured_outputs = set()  # Имена изменившихся тензоров (слева от =)
        self._autograd_ctx=autograd_ctx
        # Автодетекция размера варпа
        dummy_code = "__kernel void dummy() {}"
        dummy_prg = cl.Program(self.ctx, dummy_code).build()
        try:
            self.warp_size = dummy_prg.dummy.get_work_group_info(
                cl.kernel_work_group_info.PREFERRED_WORK_GROUP_SIZE_MULTIPLE,
                self.queue.device
            )
        except Exception:
            self.warp_size = 32

```

Я решил сохранить autograde ссылку на уровне класса чтобы можно было делать несколько последовательных вызовов localRunner.__call2__ не рискуя потерять ссылку на хранилище градиентов.



```
```

```
                raise ValueError(f"Ошибка валидации: Строка '{line}' не содержит ни одного тензора.")

            target_tensor_name = None

            # Шаг 2: Проверяем наличие знака равенства "="
            if '=' not in cleaned_line:
                # Если знака равенства нет, должно быть строго ++ или --
                if '++' not in cleaned_line and '--' not in cleaned_line:
                    raise SyntaxError(f"Ошибка валидации: Строка '{line}' не содержит знака '=' или инкремента/декремента (++ / --).")

                # Извлекаем имя тензора, к которому применен инкремент/декремент
                match_inc_dec = re.search(r'\b([a-zA-Z_][a-zA-Z0-9_]*)\b\s*(?:\+\+|--)', cleaned_line)
                if match_inc_dec:
                    target_tensor_name = match_inc_dec.group(1)
            else:
                # Шаг 3: Разделяем строку по знаку равенства
                left_part, right_part = cleaned_line.split('=', 1)
                left_part = left_part.strip()
                right_part = right_part.strip()

                # Проверяем комбинированные операторы присваивания (+=, -=, *=, /=, %=)
                # Если перед '=' стоит один из знаков арифметики
                if left_part and left_part[-1] in ['+', '-', '*', '/', '%']:
                    target_tensor_name = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', left_part)[-1]
                else:
                    # Шаг 4: Чистое присваивание '=', проверяем правую часть на валидность данных
                    target_tensor_name = left_part

                    # Извлекаем все буквенные токены из правой части, кроме вызовов функций nn_
                    right_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                    for rt in right_tokens:
                        if rt.startswith('nn_') or rt in ['sigmoid', 'tanh', 'dropout', 'tid']:
                            continue
                        # Если в правой части осталась строка, которая не является тензором и не была вплавлена как число
                        if rt in caller_locals and isinstance(caller_locals[rt], Tensor):
                            continue
                        raise TypeError(f"Ошибка валидации: В правой части выражения '{line}' обнаружено недопустимое строковое значение '{rt}'.")

            if not target_tensor_name:
                raise ValueError(f"Ошибка валидации: Не удалось определить целевой тензор в строке '{line}'.")

            self._captured_outputs.add(target_tensor_name)
            # --- ГЕНЕРАЦИЯ СТРОКИ ДЛЯ BACKWARD ПО ТВОЕМУ ПЛАНУ ---
            if self.is_training:
                # Дублируем очищенную строку (например, Y=nn_tanh(X))
                bwd_line = cleaned_line

                if "nn_dropout" in bwd_line:
                    # Исключение для dropout: заменяем функцию на nn_dropout_backward,
                    # заменяя первый аргумент X на его градиент grad_X (входной градиент идет вместо X)
                    # Формат: nn_dropout_backward(grad_Y, p, tid, seed)
                    bwd_line = bwd_line.replace("nn_dropout", "nn_dropout_backward")
                    # Заменяем целевой таргет на градиент источника
                    compiled_bwd = f"if (tid < {target_tensor_name}Size) {{ grad_{target_tensor_name}[tid] += {bwd_line.split('=', 1)[1]}; }}"
                else:
```

```
                    # Заменяем целевой таргет на градиент источника
                    compiled_bwd = f"if (tid < {target_tensor_name}Size) {{ grad_{target_tensor_name}[tid] += {bwd_line.split('=', 1)[1]}; }}"
                else:
                    # Стандартный паттерн: ищем буквы между "nn_" и "("
                    match_func = re.search(r'nn_([a-zA-Z0-9_]+)\(', bwd_line)
                    if match_func:
                        func_name = match_func.group(1)  # получили к примеру 'tanh'
                        # Вставляем _backward перед скобкой
                        bwd_line = bwd_line.replace(f"nn_{func_name}(", f"nn_{func_name}_backward(")

                        # Находим аргумент внутри скобок, чтобы узнать имя входного тензора
                        match_args = re.search(r'nn_[a-zA-Z0-9_]+_backward\((.*?)\)', bwd_line)
                        if match_args:
                            inner_arg = match_args.group(1).split(',')[0].strip() # имя тензора X
                            # Перед закрывающей скобкой вставляем , grad_<Имя таргета>
                            # Математически: grad_X += nn_tanh_backward(X, grad_Y)
                            bwd_line = bwd_line.replace(")", f", grad_{target_tensor_name})")

                            # Перестраиваем выражение: целевым становится grad_ВходногоТензора, операция +=
                            compiled_bwd = f"if (tid < {inner_arg}Size) {{ grad_{inner_arg} += {bwd_line.split('=', 1)[1]}; }}"
                    else:
                        # Если это базовая операция типа унарного/бинарного плюс/минус/умножить без функций nn_
                        # Реализуем фолбэк накопления градиента ошибки
                        compiled_bwd = f"if (tid < {target_tensor_name}Size) {{ grad_{target_tensor_name}[tid] += grad_{target_tensor_name}[tid]; }}"

            compiled_backward_statements.append(compiled_bwd)
        # === РЕАЛИЗАЦИЯ УМНОГО ВЕКТОРНОГО ФИЛЬТРА ЗАМЕН ДЛЯ ТЕНЗОРОВ ===
        # Набор операторов и символов из вашего ТЗ (включая пробел)
        operators = [' ', '-', '+', '=', ',', '(', '/', '%', ')']

        # Обходим уникальные имена тензоров, обнаруженные в строке
        for t in detected_tensors_set:
            # 1. Формируем вектор пар: имя + оператор/символ
            pattern_vector = [f"{t}{op}" for op in operators]

            # 2. Формируем параллельный вектор замен: имя[tid] + оператор/символ
            replacement_vector = [f"{t}[tid]{op}" for op in operators]

            # 3. Выполняем алгоритм контекстной замены:
            # Пока комбинация имя+оператор встречается в строке, производим замену
            for pattern, replacement in zip(pattern_vector, replacement_vector):
                while pattern in cleaned_line:
                    cleaned_line = cleaned_line.replace(pattern, replacement)

        # Оборачиваем валидную строку в Си-конструкцию с проверкой границ
        compiled_statement = f"if (tid < {target_tensor_name}Size) {{ {cleaned_line}; }}"
        compiled_statements.append(compiled_statement)

        # Собираем входные тензоры-аргументы (все задействованные тензоры минус те, что перезаписаны)
        for t in detected_tensors_set:
            if t in caller_locals and caller_locals[t].grad_buffer is not None:
                if t not in self._captured_outputs:
                    self._captured_inputs.add(t)

        # --- СБОРКА ФИНАЛЬНОГО ТЕКСТА КЕРНЕЛА ---
        kernel_hash = abs(hash(program_text))
```

```
        compiled_statements.append(compiled_statement)

        # Собираем входные тензоры-аргументы (все задействованные тензоры минус те, что перезаписаны)
        for t in detected_tensors_set:
            if t in caller_locals and caller_locals[t].grad_buffer is not None:
                if t not in self._captured_outputs:
                    self._captured_inputs.add(t)

        # --- СБОРКА ФИНАЛЬНОГО ТЕКСТА КЕРНЕЛА ---
        kernel_hash = abs(hash(program_text))
        kernel_name = f"text_forward_{kernel_hash}"

        # Формируем список аргументов ядра динамически на основе обнаруженных тензоров
        kernel_args = []
        cl_buffers_call_fwd = []
        max_elements = 1

        for tensor_name in sorted(list(detected_tensors_set)):
            tensor_obj = caller_locals[tensor_name]
            if tensor_obj.size > max_elements:
                max_elements = tensor_obj.size

            # Добавляем указатель на буфер данных и переменную размера
            kernel_args.append(f"__global float* {tensor_name}")
            kernel_args.append(f"const int {tensor_name}Size")
            cl_buffers_call_fwd.append(tensor_obj.data_buffer)
            cl_buffers_call_fwd.append(np.int32(tensor_obj.size))

        args_str = ", ".join(kernel_args)
        forward_statements_str = "\n    ".join(compiled_statements)

        # Собираем монолитный Си-код
        full_forward_code = f"""
{BASE_CL_FUNCTIONS}

__kernel void {kernel_name}({args_str}) {{
    int tid = {self.warp_size} * get_global_id(0) + get_global_id(1);
    {forward_statements_str}
}}
"""
        # Сборка сетки потоков и выполнение на GPU
        total_count = int(np.ceil(max_elements / float(self.warp_size)) * self.warp_size)
        global_size = (total_count // self.warp_size, self.warp_size)

        program = JITProgramCache.get_or_compile(self.ctx, full_forward_code)
        kernel = getattr(program, kernel_name)
        kernel(self.queue, global_size, None, *cl_buffers_call_fwd)
        self.queue.finish()
```

```
# Создаем ленивый callback (event)
def runner_step_backward_callback():
    bwd_program = JITProgramCache.get_or_compile(self.ctx, captured_code)
    bwd_kernel = getattr(bwd_program, backward_kernel_name)
    # Выполняем вычисление градиентов на GPU по сохраненным физическим адресам буферов
    bwd_kernel(self.queue, captured_global_size, None, *captured_buffers)
    self.queue.finish()
    # Передаем ссылку на callback в менеджер графа (AutogradContext модели)
    # Вытягиваем контекст автограда из переданного аргумента или из выходного тензора
    active_ctx = autograd_ctx
    if active_ctx is None:
        for out_name in self._captured_outputs:
            t_obj = caller_locals[out_name]
            if hasattr(t_obj, 'autograd_ctx') and t_obj.autograd_ctx is not None:
                active_ctx = t_obj.autograd_ctx
                break
    if active_ctx is not None:
        active_ctx.register_backward(runner_step_backward_callback)
    else:
    # Резервный локальный стек на случай изолированных тестов
        self.backward_stack.append((captured_code, backward_kernel_name, captured_buffers, captured_global_size))
```

ОБЯЗАТЕЛЬНО УКАЗЫВАЙ НОМЕРА СТРОК КОТОРЫЕ НУЖНО ИЗМЕНИТЬ! А ТАКЖЕ НОМЕР 1 И НОМЕР ПОСМЛЕДНЕЙ СТРОК КОТОРЫЕ ОСТАЮТЬСЯ КАК ЕСТЬ!